Build Foto AI Agent from Scratch - Complete Prompt

Give this entire prompt to any AI assistant to build the complete Foto AI Agent system from the ground up.

PROJECT BRIEF

What You're Building

Foto - An autonomous AI photography coach that proactively sends personalized daily challenges based on the user's schedule, location, weather, and skill level. Think of it as a personal photography mentor that works 24/7 in the background.

The Vision

Instead of a passive chatbot waiting for user questions, Foto is an **intelligent agent** that:
- Monitors the user's schedule (work days vs. weekend vs. work-from-home)
- Checks weather conditions and sends golden hour alerts
- Generates context-appropriate challenges (complex 4-hour projects for weekends, 15-minute micro-challenges for lunch breaks)
- Tracks progress and adapts difficulty automatically
- Sends weekly reports with insights
- Learns from user behavior (completion rates, preferences, skill progression)

USER PROFILE (CRITICAL CONTEXT)

The system is being built for **Seun**, a photographer with this specific profile:

Schedule & Availability

```
{
  // Work Schedule
  workDays: "Monday-Friday, 9am-5pm (office job)",
  workFromHome: "Variable (some days WFH, detected from calendar)",

  // Photography Time
  primaryPhotoDay: "Saturday (4-8 hours dedicated photography time)",
  sundayPhotography: "Light/optional (1-2 hours if weather is good)",
  lunchBreaks: "Weekdays, 20-30 minutes (quick micro-challenges)",
  afterWork: "Sometimes (45-90 mins if golden hour timing works)",

  // Time Constraints
  morningWorkdays: "No time (getting ready for work)",
  duringWork: "No disruptions (except lunch break alerts)",
  evenings: "Limited (only if special conditions like golden hour)"
}
```

Photography Gear

```
{
  mainCamera: "Fujifilm X-T5 (weekends, serious shoots)",
  everydayCamera: "Ricoh GR IIIx (pocketable, lunch breaks)",
  filmCameras: ["Nikon FE2", "Pentax ME"],

  // Gear Usage Pattern
  saturday: "Bring X-T5 or film camera",
  lunchBreak: "Use Ricoh GR IIIx (already in pocket)",
  afterWork: "Usually Ricoh GR IIIx"
}
```

Photography Interests & Skill

```
{
  skillLevel: "Intermediate",
  interests: ["street photography", "portraits", "architecture", "urban landscapes"],
  currentFocus: "Improving composition and understanding light",
  weaknesses: ["Long exposure", "night photography", "flash photography"]
}
```

Location

- **Primary location**: Lagos, Nigeria
- **Use OpenWeatherMap API** for weather data
- Weather matters for challenge generation (don’t send outdoor challenges in heavy rain)

TECHNICAL ARCHITECTURE

Tech Stack

Frontend

- **Next.js 14** (App Router)
- **TypeScript**
- **Tailwind CSS** + **shadcn/ui** components
- **React Hook Form** + **Zod** validation
- **Recharts** for analytics

Backend

- **Next.js API Routes**
- **NextAuth.js** for authentication
- **Prisma ORM** (with PostgreSQL)
- **Scheduled Tasks** (daemon management or Vercel Cron)

External Services

- **Abacus.AI LLM API** (for challenge generation)
- **OpenWeatherMap API** (weather data)
- **SendGrid** (email notifications)
- **Google Calendar API** (optional - WFH detection)

Deployment

- **Vercel** or similar Next.js host
- **PostgreSQL** database (already provisioned)
- Domain: `myfoto.topues.com`

DATABASE SCHEMA (PRISMA)

Create `/prisma/schema.prisma`:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// User profile and authentication
model User {
  id            String    @id @default(cuid())
  email         String    @unique
  password      String
  name          String?
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt

  profile       Profile?
  preferences   Preferences?
  gear          Gear[]
  challenges    Challenge[]
  submissions   ChallengeSubmission[]
  progress      Progress?

  @@map("users")
}

// User profile information
model Profile {
  id            String    @id @default(cuid())
  userId        String    @unique
  user          User      @relation(fields: [userId], references: [id], onDelete: Cas-
cade)

  location      String?   // "Lagos, Nigeria"
  timezone      String    @default("Africa/Lagos")
  skillLevel    String    @default("intermediate") // beginner, intermediate, advanced
  biography     String?
  avatarUrl     String?

  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt

  @@map("profiles")
}

// User preferences and schedule
model Preferences {
  id                    String    @id @default(cuid())
  userId                String    @unique
  user                  User      @relation(fields: [userId], references: [id],
onDelete: Cascade)

  // Work Schedule
  workDays              Json      @default("[1,2,3,4,5]") // 0=Sunday, 6=Saturday
  workStartHour         Int       @default(9)
  workEndHour           Int       @default(17)
  workLocation          String    @default("office") // office, hybrid, remote

  // Photography Availability
  primaryPhotoDay       Int       @default(6) // Saturday
```

```
  enableLunchChallenges Boolean   @default(true)
  lunchBreakDuration    Int       @default(30) // minutes
  enableAfterWork       Boolean   @default(true)
  enableWeekendChallenges Boolean @default(true)

  // Challenge Preferences
  difficultyLevel       String    @default("adaptive") // easy, medium, hard, adaptive
  preferredSubjects     Json      @default("[\"street\",\"portrait\",\"architec-
ture\"]")
  avoidSubjects         Json      @default("[]")
  maxChallengesPerWeek  Int       @default(7)

  // Notifications
  enableEmail           Boolean   @default(true)
  enablePush            Boolean   @default(false)
  notificationTime      String    @default("morning") // morning, afternoon, evening

  // Integrations
  googleCalendarEnabled Boolean   @default(false)
  googleCalendarId      String?

  createdAt             DateTime  @default(now())
  updatedAt             DateTime  @updatedAt

  @@map("preferences")
}

// User's photography gear
model Gear {
  id            String    @id @default(cuid())
  userId        String
  user          User      @relation(fields: [userId], references: [id], onDelete: Cas-
cade)

  type          String    // camera, lens, accessory
  brand         String    // Fujifilm, Ricoh, Nikon, etc.
  model         String    // X-T5, GR IIIx, FE2, etc.
  isPrimary     Boolean   @default(false)
  usageContext  String?   // "weekends", "everyday", "special occasions"

  createdAt     DateTime  @default(now())

  @@map("gear")
}

// Generated photography challenges
model Challenge {
  id            String    @id @default(cuid())
  userId        String
  user          User      @relation(fields: [userId], references: [id], onDelete: Cas-
cade)

  title         String
  description   String    @db.Text
  type          String    // WEEKEND_DEEP_DIVE, LUNCH_MICRO, AFTER_WORK,
WORK_FROM_HOME, SUNDAY_LIGHT
  difficulty    String    // easy, medium, hard
  category      String    // street, portrait, landscape, etc.

  estimatedDuration Int   // minutes
  suggestedGear     Json  // ["Fujifilm X-T5", "23mm lens"]
  suggestedLocation String?
  weatherCondition  String? // "sunny", "cloudy", "fog", etc.
```

```
  tips          Json      // ["Tip 1", "Tip 2"]
  learningGoals Json      // ["composition", "light"]

  scheduledFor  DateTime
  expiresAt     DateTime?
  status        String    @default("pending") // pending, active, completed, skipped,
expired

  // Context at generation time
  generationContext Json  // {dayType, weather, isWFH, etc.}

  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt

  submissions   ChallengeSubmission[]

  @@index([userId, status])
  @@index([userId, scheduledFor])
  @@map("challenges")
}

// User submissions for challenges
model ChallengeSubmission {
  id            String    @id @default(cuid())
  challengeId   String
  challenge     Challenge @relation(fields: [challengeId], references: [id], onDelete:
Cascade)
  userId        String
  user          User      @relation(fields: [userId], references: [id], onDelete: Cas-
cade)

  photoUrl      String?   // S3 or cloud storage URL
  notes         String?   @db.Text
  completedAt   DateTime  @default(now())

  // AI Feedback
  aiFeedback    String?   @db.Text
  feedbackSentiment String? // positive, constructive, encouraging
  improvementAreas Json?   // ["composition", "exposure"]

  createdAt     DateTime  @default(now())

  @@map("challenge_submissions")
}

// User progress tracking
model Progress {
  id                    String    @id @default(cuid())
  userId                String    @unique
  user                  User      @relation(fields: [userId], references: [id],
onDelete: Cascade)

  // Streak tracking
  currentStreak         Int       @default(0)
  longestStreak         Int       @default(0)
  lastCompletionDate    DateTime?

  // Completion stats
  totalChallenges       Int       @default(0)
  completedChallenges   Int       @default(0)
  skippedChallenges     Int       @default(0)
```

```
  // Skill progression
  skillLevel            String    @default("intermediate")
  strongSubjects        Json      @default("[]") // ["street", "portrait"]
  improvingSubjects     Json      @default("[]")
  strugglingSubjects    Json      @default("[]")

  // Adaptive learning
  currentDifficulty     String    @default("medium")
  completionRate        Float     @default(0.0)
  averageCompletionTime Int       @default(0) // minutes

  createdAt             DateTime  @default(now())
  updatedAt             DateTime  @updatedAt

  @@map("progress")
}

// Agent action logs (for debugging and insights)
model AgentLog {
  id            String    @id @default(cuid())
  userId        String?

  action        String    // "challenge_generated", "email_sent", "streak_updated"
  details       Json      // Full context of what happened
  success       Boolean   @default(true)
  errorMessage  String?

  createdAt     DateTime  @default(now())

  @@index([userId, createdAt])
  @@index([action, createdAt])
  @@map("agent_logs")
}
```

AI AGENT INTELLIGENCE LOGIC

Core Decision Engine

The AI Agent makes decisions based on multiple factors. Here’s the pseudocode:

```
// File: /lib/challenge-generator.ts

interface ChallengeContext {
  dayOfWeek: number; // 0-6
  timeOfDay: string; // morning, afternoon, evening
  isWorkDay: boolean;
  isWFH: boolean;
  weather: WeatherData;
  userPreferences: Preferences;
  recentProgress: Progress;
}

async function generateDailyChallenge(userId: string): Promise<Challenge> {
  // 1. Gather context
  const context = await analyzeContext(userId);

  // 2. Determine challenge type
  const challengeType = determineChallengeType(context);

  // 3. Check if challenge should be sent
  if (!shouldSendChallenge(context, challengeType)) {
    return null; // Skip today
  }

  // 4. Generate challenge using AI
  const challenge = await generateWithAI(userId, context, challengeType);

  // 5. Store in database
  const savedChallenge = await prisma.challenge.create({ data: challenge });

  // 6. Send notification
  await sendChallengeNotification(userId, savedChallenge);

  // 7. Log action
  await logAgentAction('challenge_generated', { userId, challengeType, context });

  return savedChallenge;
}

function determineChallengeType(context: ChallengeContext): ChallengeType {
  const { dayOfWeek, isWorkDay, isWFH, weather, userPreferences } = context;

  // Saturday = Main photography day
  if (dayOfWeek === 6) {
    return 'WEEKEND_DEEP_DIVE';
  }

  // Sunday = Light optional challenge
  if (dayOfWeek === 0 && userPreferences.enableWeekendChallenges) {
    // Only if weather is good
    if (weather.condition === 'Clear' || weather.condition === 'Partly Cloudy') {
      return 'SUNDAY_LIGHT';
    }
    return null; // Skip if weather is bad
  }

  // Weekday + Work from home
  if (isWorkDay && isWFH) {
    return 'WORK_FROM_HOME';
  }

  // Weekday lunch break
```

```
  if (isWorkDay && context.timeOfDay === 'lunchTime' && userPrefer-
ences.enableLunchChallenges) {
    // Skip if weather is bad (outdoor challenge)
    if (weather.condition === 'Rain' || weather.condition === 'Thunderstorm') {
      return null;
    }
    return 'LUNCH_MICRO';
  }

  // After work (only if golden hour conditions)
  if (isWorkDay && context.timeOfDay === 'afternoon' && isGoldenHourOpportun-
ity(context)) {
    return 'AFTER_WORK';
  }

  return null; // No challenge today
}

function shouldSendChallenge(context: ChallengeContext, type: ChallengeType): boolean
{
  if (!type) return false;

  // Check if user has already completed today's challenge
  const hasActiveChallenge = await hasActiveChallengeToday(context.userId);
  if (hasActiveChallenge) return false;

  // Check max challenges per week
  const weekCount = await getChallengesThisWeek(context.userId);
  if (weekCount >= context.userPreferences.maxChallengesPerWeek) return false;

  return true;
}

async function generateWithAI(
  userId: string,
  context: ChallengeContext,
  type: ChallengeType
): Promise<ChallengeData> {

  const systemPrompt = `You are Foto, an expert photography coach. Generate a
personalized photography challenge.

User Profile:
- Skill level: ${context.userPreferences.skillLevel}
- Interests: ${context.userPreferences.preferredSubjects}
- Location: ${context.location}
- Available time: ${getChallengeTypeDuration(type)}
- Current weather: ${context.weather.condition}, ${context.weather.temp}°F

Challenge Type: ${type}
${getChallengeTypeDescription(type)}

Generate a JSON response with:
{
  "title": "Short catchy title",
  "description": "Detailed description (2-3 paragraphs)",
  "difficulty": "easy|medium|hard",
  "category": "street|portrait|landscape|etc",
  "estimatedDuration": 60,
  "suggestedGear": ["Fujifilm X-T5", "23mm lens"],
  "suggestedLocation": "Specific location suggestion",
  "tips": ["Tip 1", "Tip 2", "Tip 3"],
  "learningGoals": ["composition", "lighting"]
```

```
}`;

  const response = await fetch('https://apps.abacus.ai/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${process.env.ABACUSAI_API_KEY}`
    },
    body: JSON.stringify({
      model: 'gpt-4.1-mini',
      messages: [
        { role: 'system', content: systemPrompt },
        { role: 'user', content: `Generate a ${type} challenge for today.` }
      ],
      response_format: { type: "json_object" },
      max_tokens: 1500
    })
  });

  const data = await response.json();
  return JSON.parse(data.choices[0].message.content);
}
```

Challenge Type Specifications

```
const CHALLENGE_TYPES = {
  WEEKEND_DEEP_DIVE: {
    name: 'Weekend Deep Dive',
    duration: '4-8 hours',
    complexity: 'high',
    description: 'Complex project with multiple shots, location scouting, planning',
    sendTime: 'Saturday 7:00 AM',
    examples: [
      'Photo essay: Document a neighborhood transformation',
      'Portrait series: 5 strangers, one story',
      'Architecture study: Lines, shapes, shadows in the city'
    ]
  },

  LUNCH_MICRO: {
    name: 'Lunch Break Micro-Challenge',
    duration: '15-25 minutes',
    complexity: 'low',
    description: 'Single shot, nearby location, pocketable camera',
    sendTime: 'Weekdays 11:00 AM',
    examples: [
      'Capture one perfect shadow play',
      'Find geometry in architecture near your office',
      'Street candid: One moment, one frame'
    ]
  },

  WORK_FROM_HOME: {
    name: 'WFH Flexible Challenge',
    duration: '30-60 minutes',
    complexity: 'medium',
    description: 'Flexible timing, neighborhood exploration, creative indoor options',
    sendTime: 'WFH days 9:00 AM',
    examples: [
      'Morning light in your home',
      'Quick neighborhood walk: Find 3 interesting compositions',
      'Macro photography: Everyday objects transformed'
    ]
  },

  AFTER_WORK: {
    name: 'Golden Hour Alert',
    duration: '45-90 minutes',
    complexity: 'medium',
    description: 'Weather-dependent, sent only if golden hour + good conditions',
    sendTime: '1.5-2 hours before sunset',
    examples: [
      'Golden hour portraits in natural light',
      'Urban landscapes bathed in warm light',
      'Silhouettes and long shadows'
    ]
  },

  SUNDAY_LIGHT: {
    name: 'Sunday Light Challenge',
    duration: '1-2 hours',
    complexity: 'low-medium',
    description: 'Optional, relaxed, only if weather is nice',
    sendTime: 'Sunday 8:00 AM',
    examples: [
      'Morning walk: Quiet streets',
      'Café photography: People and places',
```

```
      'Experiment with a new technique'
    ]
  }
};
```

EXTERNAL API INTEGRATIONS

1. Weather API (OpenWeatherMap)

```
// File: /lib/weather.ts

interface WeatherData {
  condition: string; // "Clear", "Rain", "Clouds", "Fog"
  temp: number;
  humidity: number;
  windSpeed: number;
  sunrise: string;
  sunset: string;
  goldenHourStart: string;
  goldenHourEnd: string;
}

export async function getWeather(location: string): Promise<WeatherData> {
  const apiKey = process.env.OPENWEATHERMAP_API_KEY;

  // Geocode location first
  const geoResponse = await fetch(
    `http://api.openweathermap.org/geo/1.0/direct?q=${location}&limit=1&appid=$
{apiKey}`
  );
  const [geoData] = await geoResponse.json();

  // Get weather forecast
  const weatherResponse = await fetch(
    `https://api.openweathermap.org/data/2.5/forecast?lat=${geoData.lat}&lon=${geoData
.lon}&appid=${apiKey}&units=imperial`
  );
  const weatherData = await weatherResponse.json();

  return {
    condition: weatherData.list[0].weather[0].main,
    temp: weatherData.list[0].main.temp,
    humidity: weatherData.list[0].main.humidity,
    windSpeed: weatherData.list[0].wind.speed,
    sunrise: new Date(weatherData.city.sunrise * 1000).toISOString(),
    sunset: new Date(weatherData.city.sunset * 1000).toISOString(),
    goldenHourStart: calculateGoldenHourStart(weatherData.city.sunset),
    goldenHourEnd: new Date(weatherData.city.sunset * 1000).toISOString()
  };
}

export function isGoodPhotoWeather(weather: WeatherData): boolean {
  const badConditions = ['Rain', 'Thunderstorm', 'Drizzle'];
  return !badConditions.includes(weather.condition);
}

export function isGoldenHourOpportunity(weather: WeatherData): boolean {
  const now = new Date();
  const goldenHourStart = new Date(weather.goldenHourStart);
  const goldenHourEnd = new Date(weather.goldenHourEnd);

  // Check if we're within 1.5-2 hours before sunset
  const hoursUntilGoldenHour = (goldenHourStart.getTime() - now.getTime()) / (1000 * 6
0 * 60);

  return hoursUntilGoldenHour >= 1.5 && hoursUntilGoldenHour <= 2 && isGoodPhotoWeath-
er(weather);
}
```

2. Email Service (SendGrid)

```
// File: /lib/email.ts

import sgMail from '@sendgrid/mail';

sgMail.setApiKey(process.env.SENDGRID_API_KEY);

export async function sendChallengeEmail(user: User, challenge: Challenge) {
  const template = getChallengeEmailTemplate(challenge.type);

  const msg = {
    to: user.email,
    from: 'coach@myfoto.com',
    subject: `📸 ${challenge.title}`,
    html: template(user, challenge)
  };

  await sgMail.send(msg);
}

function getChallengeEmailTemplate(type: ChallengeType) {
  // Return appropriate HTML template based on challenge type
  // Templates included in next section
}
```

Email Templates

Saturday Challenge Email:

```
<div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
  <h1 style="color: #2563eb;">☀️ Your Saturday Photography Challenge</h1>

  <div style="background: #f3f4f6; padding: 20px; border-radius: 8px; margin: 20px
0;">
    <h2>{{challenge.title}}</h2>
    <p style="font-size: 16px; line-height: 1.6;">{{challenge.description}}</p>
  </div>

  <div style="margin: 20px 0;">
    <h3>📋 Challenge Details</h3>
    <ul>
      <li><strong>Duration:</strong> 4-8 hours</li>
      <li><strong>Difficulty:</strong> {{challenge.difficulty}}</li>
      <li><strong>Suggested Gear:</strong> {{challenge.suggestedGear.join(', ')}}</li>
      <li><strong>Location Idea:</strong> {{challenge.suggestedLocation}}</li>
    </ul>
  </div>

  <div style="background: #dbeafe; padding: 15px; border-left: 4px solid #2563eb;
margin: 20px 0;">
    <h3>💡 Tips</h3>
    <ul>
      {{#each challenge.tips}}
        <li>{{this}}</li>
      {{/each}}
    </ul>
  </div>

  <div style="text-align: center; margin: 30px 0;">
    <a href="https://myfoto.topues.com/dashboard"
       style="background: #2563eb; color: white; padding: 15px 30px; text-decoration:
none; border-radius: 6px; display: inline-block;">
      View Full Challenge
    </a>
  </div>

  <div style="margin-top: 30px; padding-top: 20px; border-top: 1px solid #e5e7eb;
font-size: 14px; color: #6b7280;">
    <p>Weather today: {{weather.condition}}, {{weather.temp}}°F</p>
    <p>Golden hour: {{weather.goldenHourStart}} - {{weather.goldenHourEnd}}</p>
  </div>
</div>
```

Lunch Break Challenge Email:

```
<div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
  <h1 style="color: #2563eb;">🍽️ Quick Lunch Break Challenge</h1>

  <div style="background: #fef3c7; padding: 20px; border-radius: 8px; margin: 20px 0;
border-left: 4px solid #f59e0b;">
    <h2>{{challenge.title}}</h2>
    <p>Quick 15-minute challenge perfect for your lunch break!</p>
    <p style="font-size: 16px; line-height: 1.6;">{{challenge.description}}</p>
  </div>

  <div style="margin: 20px 0;">
    <p><strong>⏱️ Time needed:</strong> 15-25 minutes</p>
    <p><strong>📷 Gear:</strong> Ricoh GR IIIx (in your pocket!)</p>
    <p><strong>📍 Where:</strong> Near your office or lunch spot</p>
  </div>

  <div style="text-align: center; margin: 30px 0;">
    <a href="https://myfoto.topues.com/dashboard/challenges"
       style="background: #f59e0b; color: white; padding: 12px 24px; text-decoration:
none; border-radius: 6px; display: inline-block;">
      Let's Go!
    </a>
  </div>
</div>
```

3. Google Calendar API (Optional - WFH Detection)

```typescript
// File: /lib/calendar.ts

import { google } from 'googleapis';

export async function isWorkFromHomeToday(userId: string): Promise<boolean> {
  const user = await prisma.user.findUnique({
    where: { id: userId },
    include: { preferences: true }
  });

  if (!user.preferences.googleCalendarEnabled) {
    return false;
  }

  const oauth2Client = new google.auth.OAuth2(
    process.env.GOOGLE_CLIENT_ID,
    process.env.GOOGLE_CLIENT_SECRET,
    process.env.GOOGLE_REDIRECT_URI
  );

  // Get user's OAuth tokens from database
  // ... token retrieval logic

  const calendar = google.calendar({ version: 'v3', auth: oauth2Client });

  const today = new Date();
  const startOfDay = new Date(today.setHours(0, 0, 0, 0));
  const endOfDay = new Date(today.setHours(23, 59, 59, 999));

  const response = await calendar.events.list({
    calendarId: 'primary',
    timeMin: startOfDay.toISOString(),
    timeMax: endOfDay.toISOString(),
    q: 'WFH OR "work from home" OR remote'
  });

  return response.data.items && response.data.items.length > 0;
}
```

SCHEDULED TASKS

These tasks run automatically in the background:

1. Daily Challenge Generator

```
// Scheduled: Saturday 7:00 AM, Weekdays 11:00 AM, etc.

async function scheduledChallengeGeneration() {
  const users = await prisma.user.findMany({
    include: { preferences: true, profile: true }
  });

  for (const user of users) {
    try {
      await generateDailyChallenge(user.id);
    } catch (error) {
      await logAgentAction('challenge_generation_failed', {
        userId: user.id,
        error: error.message
      });
    }
  }
}
```

2. Weather Monitor

```
// Scheduled: Every 3 hours

async function monitorWeatherOpportunities() {
  const users = await prisma.user.findMany({
    include: { preferences: true, profile: true }
  });

  for (const user of users) {
    const weather = await getWeather(user.profile.location);

    // Check for golden hour opportunity
    if (isGoldenHourOpportunity(weather)) {
      await sendGoldenHourAlert(user, weather);
    }

    // Check for special conditions (fog, clear skies after rain, etc.)
    if (isSpecialPhotoCondition(weather)) {
      await sendSpecialConditionAlert(user, weather);
    }
  }
}
```

3. Weekly Report Generator

```
// Scheduled: Sunday 8:00 PM

async function generateWeeklyReport() {
  const users = await prisma.user.findMany();

  for (const user of users) {
    const weekStats = await getWeekStats(user.id);
    const report = await generateReportPDF(weekStats);
    await sendWeeklyReportEmail(user, report);
  }
}

async function getWeekStats(userId: string) {
  const oneWeekAgo = new Date(Date.now() - 7 * 24 * 60 * 60 * 1000);

  const challenges = await prisma.challenge.findMany({
    where: {
      userId,
      createdAt: { gte: oneWeekAgo }
    },
    include: { submissions: true }
  });

  return {
    totalChallenges: challenges.length,
    completed: challenges.filter(c => c.status === 'completed').length,
    skipped: challenges.filter(c => c.status === 'skipped').length,
    favoriteCategory: getMostFrequentCategory(challenges),
    streak: await getCurrentStreak(userId),
    improvementAreas: await analyzeImprovementAreas(userId)
  };
}
```

4. Streak Manager

```
// Scheduled: Daily 11:00 PM

async function updateStreaks() {
  const users = await prisma.user.findMany({
    include: { progress: true }
  });

  for (const user of users) {
    const today = new Date().toDateString();
    const hadChallengeToday = await prisma.challenge.findFirst({
      where: {
        userId: user.id,
        scheduledFor: {
          gte: new Date(today)
        },
        status: 'completed'
      }
    });

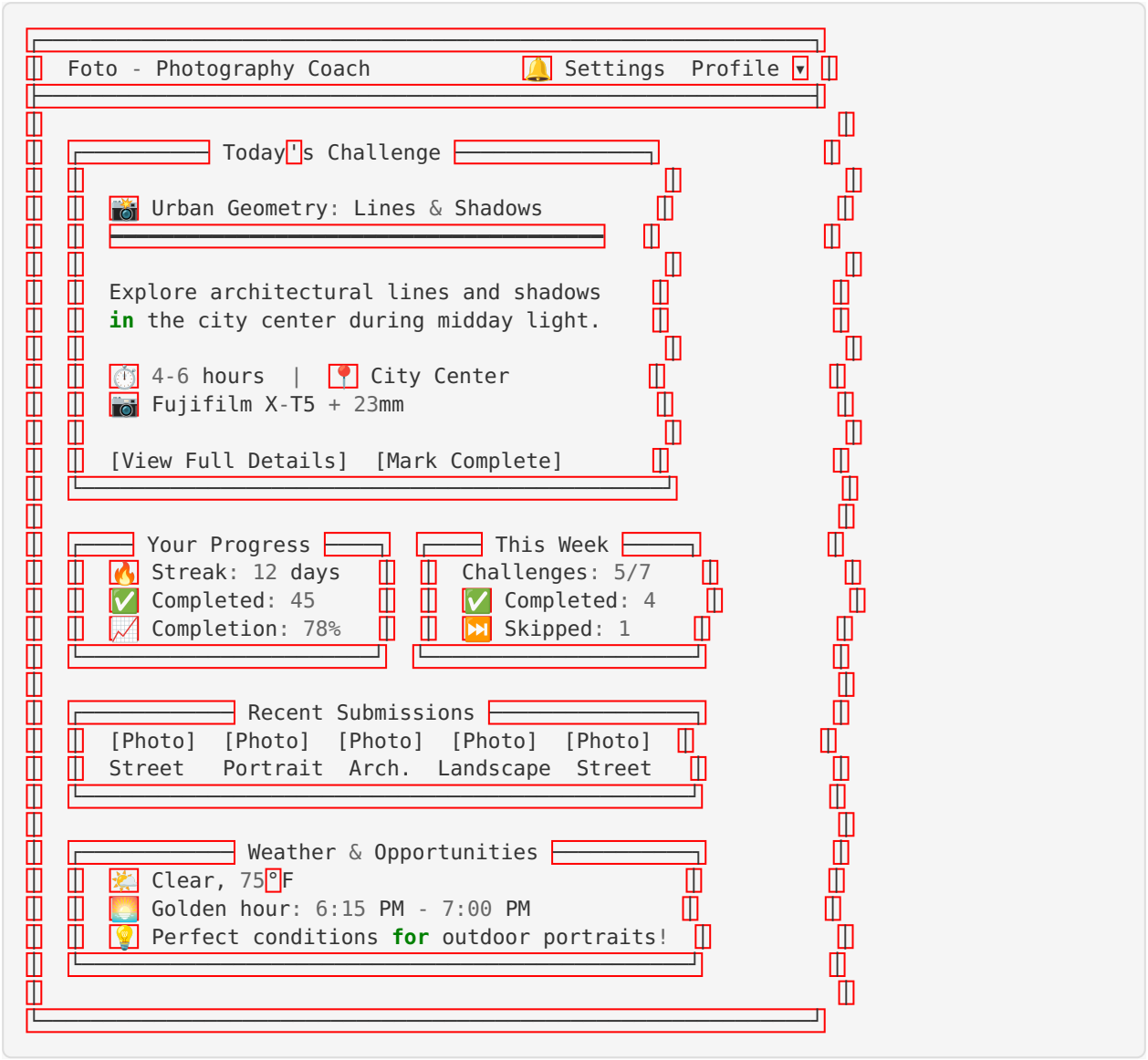
    if (hadChallengeToday) {
      // Increment streak
      await prisma.progress.update({
        where: { userId: user.id },
        data: {
          currentStreak: { increment: 1 },
          longestStreak: {
            set: Math.max(user.progress.currentStreak + 1, user.progress.longestStreak
)
          },
          lastCompletionDate: new Date()
        }
      });

      // Check for milestone
      if ((user.progress.currentStreak + 1) % 7 === 0) {
        await sendStreakMilestoneEmail(user, user.progress.currentStreak + 1);
      }
    } else {
      // Check if streak should reset
      const daysSinceLastCompletion = user.progress.lastCompletionDate
        ? (Date.now() - user.progress.lastCompletionDate.getTime()) / (1000 * 60 * 60
* 24)
        : Infinity;

      if (daysSinceLastCompletion > 1) {
        await prisma.progress.update({
          where: { userId: user.id },
          data: { currentStreak: 0 }
        });
      }
    }
  }
}
```

USER INTERFACE

Dashboard Layout



Key Pages

1. `/dashboard` - Main Dashboard

- Today's challenge card (prominent)
- Quick stats (streak, completion rate, total)
- Recent submissions gallery
- Weather widget
- Upcoming challenges preview

2. `/dashboard/challenges` - Challenges List

- Tabs: Pending, Active, Completed, Skipped
- Filter by: Type, Date, Category, Difficulty
- Each challenge shows:

- Title, category, difficulty
- Estimated time
- Status badge
- Quick actions (View, Complete, Skip)

3. `/dashboard/challenges/[id]` - Challenge Detail

- Full description
- Tips and learning goals
- Suggested gear and location
- Weather at challenge time
- Upload photo form
- Mark complete/skip buttons
- AI feedback display (after submission)

4. `/dashboard/progress` - Progress & Analytics

- Streak visualization
- Completion rate chart (by week/month)
- Category breakdown (pie chart)
- Skill progression indicators
- Favorite subjects (tags)
- Download report button

5. `/dashboard/settings` - Settings

- Work schedule configuration
- Photography availability
- Challenge preferences (difficulty, subjects, frequency)
- Notification settings
- Gear management
- Integrations (Google Calendar, Location)
- Pause/reduce challenge frequency

6. `/login` & `/signup` - Authentication

- Email/password login
- New user signup
- OAuth options (optional)

7. `/onboarding` - First-time User Wizard

- Step 1: Basic info (name, location)
- Step 2: Work schedule
- Step 3: Photography availability
- Step 4: Gear setup
- Step 5: Interests and skill level
- Step 6: Notification preferences

API ENDPOINTS

Authentication

`POST /api/auth/signup`

```
{
  "email": "sesal78@gmail.com",
  "password": "foto2024",
  "name": "Seun"
}
```

`POST /api/auth/login`

```
{
  "email": "sesal78@gmail.com",
  "password": "foto2024"
}
```

User Management

`GET /api/user/profile`

Returns: User profile + preferences + gear

`PUT /api/user/profile`

Update profile information

`PUT /api/user/preferences`

Update user preferences (schedule, notifications, etc.)

`POST /api/user/gear`

Add new gear item

`DELETE /api/user/gear/:id`

Remove gear item

Challenges

`GET /api/challenges`

Query params: `status`, `type`, `from`, `to`
Returns: List of challenges

`GET /api/challenges/:id`

Returns: Challenge details + submissions

`POST /api/challenges/generate`

Manual challenge generation (for testing)

`POST /api/challenges/:id/complete`

```
{
  "photoUrl": "https://upload.wikimedia.org/wikipedia/commons/thumb/a/ab/Sun-
set_at_Long_Beach_%28South_Africa%29.jpg/500px-Sun-
set_at_Long_Beach_%28South_Africa%29.jpg",
  "notes": "Shot this during golden hour..."
}
```

`POST /api/challenges/:id/skip`

```
{
  "reason": "weather_bad" | "no_time" | "not_interested"
}
```

`POST /api/challenges/:id/feedback`

Request AI feedback for submission

Progress

`GET /api/progress`

Returns: Full progress stats + analytics

`GET /api/progress/weekly`

Returns: Current week stats

`GET /api/progress/report`

Query params: `from`, `to`, `format` (json|pdf)

IMPLEMENTATION STEPS

Phase 1: Foundation (Days 1-2)

1.1 Database Setup

- [ ] Create Prisma schema
- [ ] Run `prisma migrate dev`
- [ ] Create seed script with test user
- [ ] Seed database

1.2 Authentication

- [ ] Set up NextAuth.js
- [ ] Create login/signup pages
- [ ] Implement session management
- [ ] Test auth flow

1.3 Basic UI

- [ ] Create dashboard layout
- [ ] Add navigation
- [ ] Create empty pages (dashboard, challenges, progress, settings)

Phase 2: Core Challenge System (Days 3-4)

2.1 Challenge Generation

- [ ] Create `/lib/challenge-generator.ts`
- [ ] Implement context analyzer
- [ ] Implement challenge type selector
- [ ] Integrate Abacus.AI LLM API
- [ ] Test challenge generation

2.2 Challenge API Endpoints

- [ ] `GET /api/challenges` - List
- [ ] `GET /api/challenges/:id` - Detail
- [ ] `POST /api/challenges/generate` - Generate
- [ ] `POST /api/challenges/:id/complete` - Complete
- [ ] `POST /api/challenges/:id/skip` - Skip

2.3 Challenge UI

- [ ] Dashboard: Today’s challenge card
- [ ] Challenges list page
- [ ] Challenge detail page
- [ ] Complete challenge form
- [ ] Upload photo functionality

Phase 3: External Integrations (Days 5-6)

3.1 Weather API

- [ ] Create `/lib/weather.ts`
- [ ] Implement OpenWeatherMap integration
- [ ] Test weather fetching
- [ ] Add weather display to dashboard

3.2 Email Service

- [ ] Create `/lib/email.ts`
- [ ] Set up SendGrid
- [ ] Create email templates
- [ ] Test email sending

3.3 Calendar (Optional)

- [ ] Set up Google Calendar OAuth
- [ ] Implement WFH detection
- [ ] Add calendar settings page

Phase 4: Scheduled Tasks (Days 7-8)

4.1 Daily Challenge Generation

- [ ] Create scheduled task
- [ ] Test on different day types
- [ ] Add logging

4.2 Weather Monitor

- [ ] Create weather monitoring task
- [ ] Implement golden hour alerts
- [ ] Test special condition detection

4.3 Weekly Reports

- [ ] Create report generation task
- [ ] Implement PDF generation
- [ ] Test email delivery

4.4 Streak Manager

- [ ] Create streak update task
- [ ] Implement milestone notifications
- [ ] Test streak logic

Phase 5: Progress & Analytics (Days 9-10)

5.1 Progress Tracking

- [ ] Create `/lib/progress.ts`
- [ ] Implement completion rate calculator
- [ ] Implement skill progression analyzer
- [ ] Implement adaptive difficulty

5.2 Progress UI

- [ ] Progress dashboard page
- [ ] Charts and visualizations
- [ ] Download report feature

5.3 Settings

- [ ] Settings page UI
- [ ] Form for all preferences
- [ ] Gear management
- [ ] Integration toggles

Phase 6: Testing & Polish (Days 11-12)

6.1 End-to-End Testing

- [ ] Test signup → onboarding → first challenge
- [ ] Test challenge completion flow
- [ ] Test all scheduled tasks

- [ ] Test email delivery
- [ ] Test weather integration

6.2 Polish

- [ ] Mobile responsiveness
- [ ] Loading states
- [ ] Error handling
- [ ] Empty states

Phase 7: Deployment

- [ ] Set up production database
- [ ] Configure environment variables
- [ ] Deploy to Vercel
- [ ] Set up scheduled tasks (Vercel Cron)
- [ ] Test in production
- [ ] Monitor logs

TESTING CHECKLIST

Authentication

- [ ] Can create new account
- [ ] Can log in
- [ ] Session persists across page reloads
- [ ] Can log out
- [ ] Password validation works

Onboarding

- [ ] Wizard displays all steps
- [ ] Can complete all steps
- [ ] Data saves correctly
- [ ] Redirects to dashboard after completion

Challenge Generation

- [ ] Saturday challenge generates correctly (complex, 4-8 hours)
- [ ] Weekday lunch challenge generates correctly (simple, 15-25 mins)
- [ ] WFH challenge generates correctly
- [ ] Sunday challenge respects weather
- [ ] Lunch challenge skips if weather is bad
- [ ] Golden hour alert triggers correctly
- [ ] Challenges respect user preferences

- [ ] Challenges adapt to skill level

Challenge Management

- [ ] Can view challenge list
- [ ] Can filter challenges
- [ ] Can view challenge details
- [ ] Can mark challenge complete
- [ ] Can upload photo
- [ ] Can skip challenge
- [ ] Challenge status updates correctly

Progress

- [ ] Streak increments on completion
- [ ] Streak resets correctly
- [ ] Completion rate calculates correctly
- [ ] Stats display accurately
- [ ] Charts render correctly
- [ ] Can download report

Settings

- [ ] Can update work schedule
- [ ] Can update photography availability
- [ ] Can add/remove gear
- [ ] Can update notification preferences
- [ ] Changes save and apply immediately

External Integrations

- [ ] Weather API returns correct data
- [ ] Emails send successfully
- [ ] Email templates render correctly
- [ ] Calendar integration works (if enabled)

Scheduled Tasks

- [ ] Daily challenge task runs
- [ ] Weather monitor task runs
- [ ] Weekly report task runs
- [ ] Streak manager task runs
- [ ] Tasks log correctly

UI/UX

- [ ] Mobile responsive

- [ ] Loading states show
- [ ] Errors display user-friendly messages
- [ ] Empty states handled
- [ ] Navigation works correctly
- [ ] All buttons functional
- [ ] Forms validate correctly

ENVIRONMENT VARIABLES

Create `.env` file:

```
# Database
DATABASE_URL="postgresql://..."

# NextAuth
NEXTAUTH_SECRET="your-secret-here"
NEXTAUTH_URL="https://myfoto.topues.com"

# Abacus.AI (LLM)
ABACUSAI_API_KEY="your-api-key"

# OpenWeatherMap
OPENWEATHERMAP_API_KEY="your-api-key"

# SendGrid
SENDGRID_API_KEY="your-api-key"
SENDGRID_FROM_EMAIL="coach@myfoto.com"

# Google Calendar (Optional)
GOOGLE_CLIENT_ID="your-client-id"
GOOGLE_CLIENT_SECRET="your-client-secret"
GOOGLE_REDIRECT_URI="https://myfoto.topues.com/api/auth/callback/google"

# AWS S3 (for photo uploads)
AWS_BUCKET_NAME="foto-uploads"
AWS_FOLDER_PREFIX="uploads/"
```

PROJECT FILE STRUCTURE

```
/home/ubuntu/foto_app/nextjs_space/
├── app/
│   ├── (auth)/
│   │   ├── login/
│   │   │   └── page.tsx
│   │   ├── signup/
│   │   │   └── page.tsx
│   │   └── onboarding/
│   │       └── page.tsx
│   ├── dashboard/
│   │   ├── page.tsx
│   │   ├── challenges/
│   │   │   ├── page.tsx
│   │   │   └── [id]/
│   │   │       └── page.tsx
│   │   ├── progress/
│   │   │   └── page.tsx
│   │   └── settings/
│   │       └── page.tsx
│   ├── api/
│   │   ├── auth/
│   │   │   └── [...nextauth]/
│   │   │       └── route.ts
│   │   ├── challenges/
│   │   │   ├── route.ts
│   │   │   └── [id]/
│   │   │       ├── route.ts
│   │   │       ├── complete/
│   │   │       │   └── route.ts
│   │   │       └── skip/
│   │   │           └── route.ts
│   │   ├── generate/
│   │   │   └── route.ts
│   │   ├── user/
│   │   │   ├── profile/
│   │   │   │   └── route.ts
│   │   │   ├── preferences/
│   │   │   │   └── route.ts
│   │   │   └── gear/
│   │   │       └── route.ts
│   │   ├── progress/
│   │   │   ├── route.ts
│   │   │   ├── weekly/
│   │   │   │   └── route.ts
│   │   │   └── report/
│   │   │       └── route.ts
│   │   └── scheduled/
│   │       ├── daily-challenge/
│   │       │   └── route.ts
│   │       ├── weather-monitor/
│   │       │   └── route.ts
│   │       ├── weekly-report/
│   │       │   └── route.ts
│   │       └── streak-manager/
│   │           └── route.ts
│   ├── layout.tsx
│   └── globals.css
├── components/
│   ├── dashboard/
│   │   ├── challenge-card.tsx
│   │   ├── stats-widget.tsx
│   │   └── weather-widget.tsx
```

```
│   │   └── recent-submissions.tsx
│   ├── challenges/
│   │   ├── challenge-list.tsx
│   │   ├── challenge-item.tsx
│   │   ├── challenge-detail.tsx
│   │   ├── complete-form.tsx
│   │   └── photo-upload.tsx
│   ├── progress/
│   │   ├── streak-display.tsx
│   │   ├── completion-chart.tsx
│   │   ├── category-breakdown.tsx
│   │   └── skill-progression.tsx
│   ├── settings/
│   │   ├── schedule-form.tsx
│   │   ├── preferences-form.tsx
│   │   ├── gear-manager.tsx
│   │   └── notification-settings.tsx
│   ├── auth/
│   │   ├── login-form.tsx
│   │   ├── signup-form.tsx
│   │   └── onboarding-wizard.tsx
│   └── ui/
│       └── (shadcn components)
├── lib/
│   ├── challenge-generator.ts
│   ├── weather.ts
│   ├── email.ts
│   ├── calendar.ts
│   ├── progress.ts
│   ├── db.ts
│   ├── utils.ts
│   └── aws-config.ts
├── prisma/
│   └── schema.prisma
├── scripts/
│   └── seed.ts
├── public/
│   └── images/
└── package.json
```

SEED DATA

Create `/scripts/seed.ts`:

```
import { PrismaClient } from '@prisma/client';
import bcrypt from 'bcryptjs';

const prisma = new PrismaClient();

async function main() {
  // Create test user
  const hashedPassword = await bcrypt.hash('foto2024', 10);

  const user = await prisma.user.create({
    data: {
      email: 'sesal78@gmail.com',
      password: hashedPassword,
      name: 'Seun',
      profile: {
        create: {
          location: 'Lagos, Nigeria',
          timezone: 'Africa/Lagos',
          skillLevel: 'intermediate',
          biography: 'Passionate photographer exploring street, portrait, and
architecture photography.'
        }
      },
      preferences: {
        create: {
          workDays: [1, 2, 3, 4, 5], // Mon-Fri
          workStartHour: 9,
          workEndHour: 17,
          workLocation: 'office',
          primaryPhotoDay: 6, // Saturday
          enableLunchChallenges: true,
          lunchBreakDuration: 30,
          enableAfterWork: true,
          enableWeekendChallenges: true,
          difficultyLevel: 'adaptive',
          preferredSubjects: ['street', 'portrait', 'architecture'],
          avoidSubjects: [],
          maxChallengesPerWeek: 7,
          enableEmail: true,
          enablePush: false,
          notificationTime: 'morning',
          googleCalendarEnabled: false
        }
      },
      gear: {
        create: [
          {
            type: 'camera',
            brand: 'Fujifilm',
            model: 'X-T5',
            isPrimary: true,
            usageContext: 'weekends'
          },
          {
            type: 'camera',
            brand: 'Ricoh',
            model: 'GR IIIx',
            isPrimary: false,
            usageContext: 'everyday'
          },
          {
            type: 'camera',
```

```
            brand: 'Nikon',
            model: 'FE2',
            isPrimary: false,
            usageContext: 'special occasions'
          },
          {
            type: 'camera',
            brand: 'Pentax',
            model: 'ME',
            isPrimary: false,
            usageContext: 'special occasions'
          }
        ]
      },
      progress: {
        create: {
          currentStreak: 0,
          longestStreak: 0,
          totalChallenges: 0,
          completedChallenges: 0,
          skippedChallenges: 0,
          skillLevel: 'intermediate',
          strongSubjects: [],
          improvingSubjects: [],
          strugglingSubjects: [],
          currentDifficulty: 'medium',
          completionRate: 0.0,
          averageCompletionTime: 0
        }
      }
    }
  });

  // Create sample challenges
  const challenges = [
    {
      userId: user.id,
      title: 'Urban Geometry: Lines & Shadows',
      description: 'Explore architectural lines and geometric patterns in the city
center...',
      type: 'WEEKEND_DEEP_DIVE',
      difficulty: 'medium',
      category: 'architecture',
      estimatedDuration: 300,
      suggestedGear: ['Fujifilm X-T5', '23mm lens'],
      suggestedLocation: 'City Center',
      weatherCondition: 'Clear',
      tips: [
        'Look for strong diagonal lines',
        'Pay attention to shadow patterns',
        'Try different angles'
      ],
      learningGoals: ['composition', 'geometry', 'light'],
      scheduledFor: new Date('2024-11-23T07:00:00'),
      status: 'completed',
      generationContext: {
        dayType: 'saturday',
        weather: 'Clear',
        isWFH: false
      }
    },
    {
      userId: user.id,
```

```
      title: 'Lunch Break: Street Candid',
      description: 'Capture one candid moment during your lunch break...',
      type: 'LUNCH_MICRO',
      difficulty: 'easy',
      category: 'street',
      estimatedDuration: 20,
      suggestedGear: ['Ricoh GR IIIx'],
      suggestedLocation: 'Near office',
      weatherCondition: 'Partly Cloudy',
      tips: [
        'Keep your camera ready',
        'Look for interesting moments',
        'Don't overthink it'
      ],
      learningGoals: ['timing', 'observation'],
      scheduledFor: new Date('2024-11-18T11:00:00'),
      status: 'completed',
      generationContext: {
        dayType: 'weekday',
        weather: 'Partly Cloudy',
        isWFH: false
      }
    }
  ];

  for (const challenge of challenges) {
    await prisma.challenge.create({ data: challenge });
  }

  console.log('✅ Seed data created successfully!');
}

main()
  .catch((e) => {
    console.error(e);
    process.exit(1);
  })
  .finally(async () => {
    await prisma.disconnect();
  });
```

Run: `yarn prisma db seed`

KEY IMPLEMENTATION NOTES

1. Challenge Generation Timing

```
// Don't send challenges at random times - be strategic:

// Saturday: 7:00 AM (user wakes up, plans the day)
// Weekday lunch: 11:00 AM (before lunch break)
// WFH days: 9:00 AM (after work starts, flexible timing)
// After work: 3:30-4:00 PM (1.5-2 hours before sunset if conditions are perfect)
// Sunday: 8:00 AM (optional, only if weather is great)
```

2. Weather-Dependent Logic

```
// ALWAYS check weather before sending outdoor challenges:

if (challengeType === 'LUNCH_MICRO' && weather.condition === 'Rain') {
  // Skip - don't send lunch challenge in rain
  return null;
}

if (challengeType === 'SUNDAY_LIGHT' && !isGoodPhotoWeather(weather)) {
  // Skip - Sunday is optional, only if weather is nice
  return null;
}

if (isGoldenHourOpportunity(weather)) {
  // Send special alert even if not scheduled
  sendGoldenHourAlert(user, weather);
}
```

3. Adaptive Difficulty

```
// Monitor completion rates and adjust:

async function adaptDifficulty(userId: string) {
  const last10Challenges = await getRecentChallenges(userId, 10);
  const completionRate = calculateCompletionRate(last10Challenges);

  if (completionRate > 0.85) {
    // User is crushing it - increase difficulty
    await increaseDifficulty(userId);
  } else if (completionRate < 0.50) {
    // User is struggling - decrease difficulty
    await decreaseDifficulty(userId);
  }
}
```

4. Photo Upload (S3)

```
// NEVER save files locally - always use S3

import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';

async function uploadPhoto(file: File, userId: string): Promise<string> {
  const s3 = new S3Client({});
  const key = `uploads/${userId}/${Date.now()}-${file.name}`;

  const buffer = Buffer.from(await file.arrayBuffer());

  await s3.send(new PutObjectCommand({
    Bucket: process.env.AWS_BUCKET_NAME,
    Key: key,
    Body: buffer,
    ContentType: file.type
  }));

  return key; // Save this in database as cloud_storage_path
}
```

5. Email Best Practices

```
// Use descriptive subject lines
// Always include weather info
// Add direct links to dashboard
// Make emails mobile-friendly
// Include unsubscribe option

const emailSubject = {
  WEEKEND_DEEP_DIVE: '📸 Your Saturday Photography Challenge',
  LUNCH_MICRO: '🍽️ Quick Lunch Break Challenge',
  WORK_FROM_HOME: '🏠 WFH Photography Challenge',
  GOLDEN_HOUR: '🌅 Perfect Golden Hour Alert!',
  WEEKLY_REPORT: '📊 Your Weekly Photography Report'
};
```

SUCCESS CRITERIA

The implementation is complete when:

✅ **User can sign up and complete onboarding**
✅ **User receives Saturday challenge at 7 AM (complex, 4-8 hours)**
✅ **User receives weekday lunch challenge at 11 AM (simple, 15-25 mins)**
✅ **System skips lunch challenge if weather is bad**
✅ **System sends golden hour alert if conditions are perfect**
✅ **User can mark challenges complete and upload photos**
✅ **AI feedback is generated for submissions**
✅ **Progress stats update in real-time**
✅ **Streak increments/resets correctly**

✅ **Weekly report emails on Sunday**
✅ **Settings allow full customization**
✅ **System adapts difficulty based on completion rates**
✅ **All scheduled tasks run reliably**
✅ **Emails send successfully**
✅ **Weather integration works**
✅ **UI is responsive and polished**

START BUILDING

Follow these phases in order:

1. **Foundation** (Database, Auth, Basic UI)
2. **Core System** (Challenge generation, APIs, UI)
3. **Integrations** (Weather, Email, Calendar)
4. **Automation** (Scheduled tasks)
5. **Analytics** (Progress tracking, adaptive learning)
6. **Testing** (End-to-end, edge cases)
7. **Deployment** (Production, monitoring)

Recommended Approach

- Work incrementally
- Test each feature before moving on
- Use the seed data to test with realistic scenarios
- Focus on the user experience (Seun's schedule and preferences)
- Keep the agent logic simple and deterministic
- Log everything for debugging

Questions to Ask

If you need clarification:
- About specific implementation details
- About design choices
- About external service setup
- About testing strategies
- About deployment

Now build the Foto AI Agent! 🚀📸

Good luck! This is a comprehensive system but following the phases will make it manageable. Focus on getting the core working first (challenge generation + UI), then add the bells and whistles (email, weather, analytics).

Remember: The goal is a proactive AI that understands Seun’s schedule and sends the right challenge at the right time. Everything else supports that core mission.